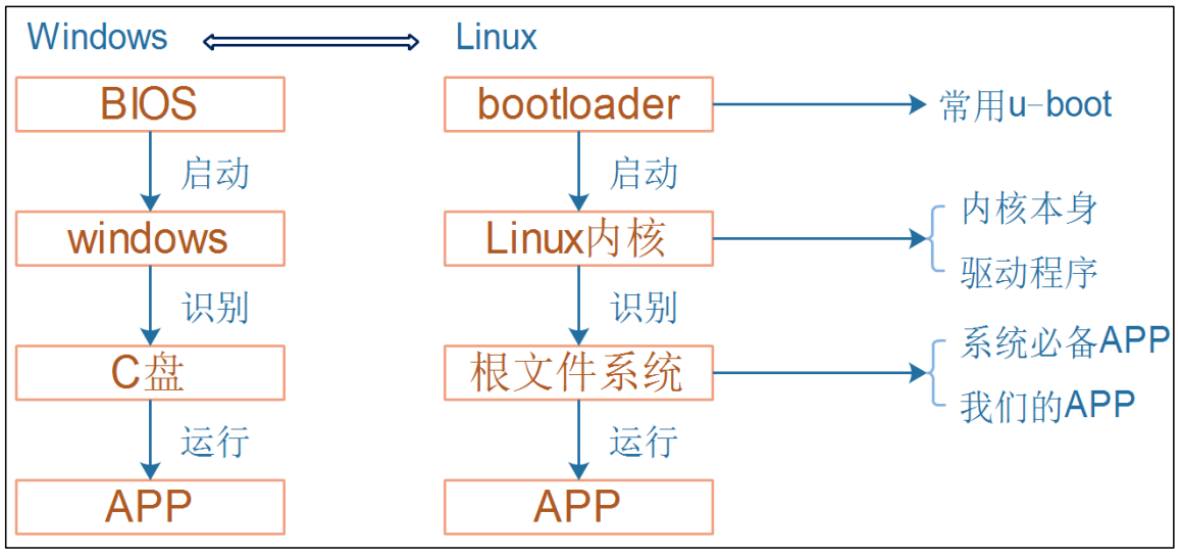
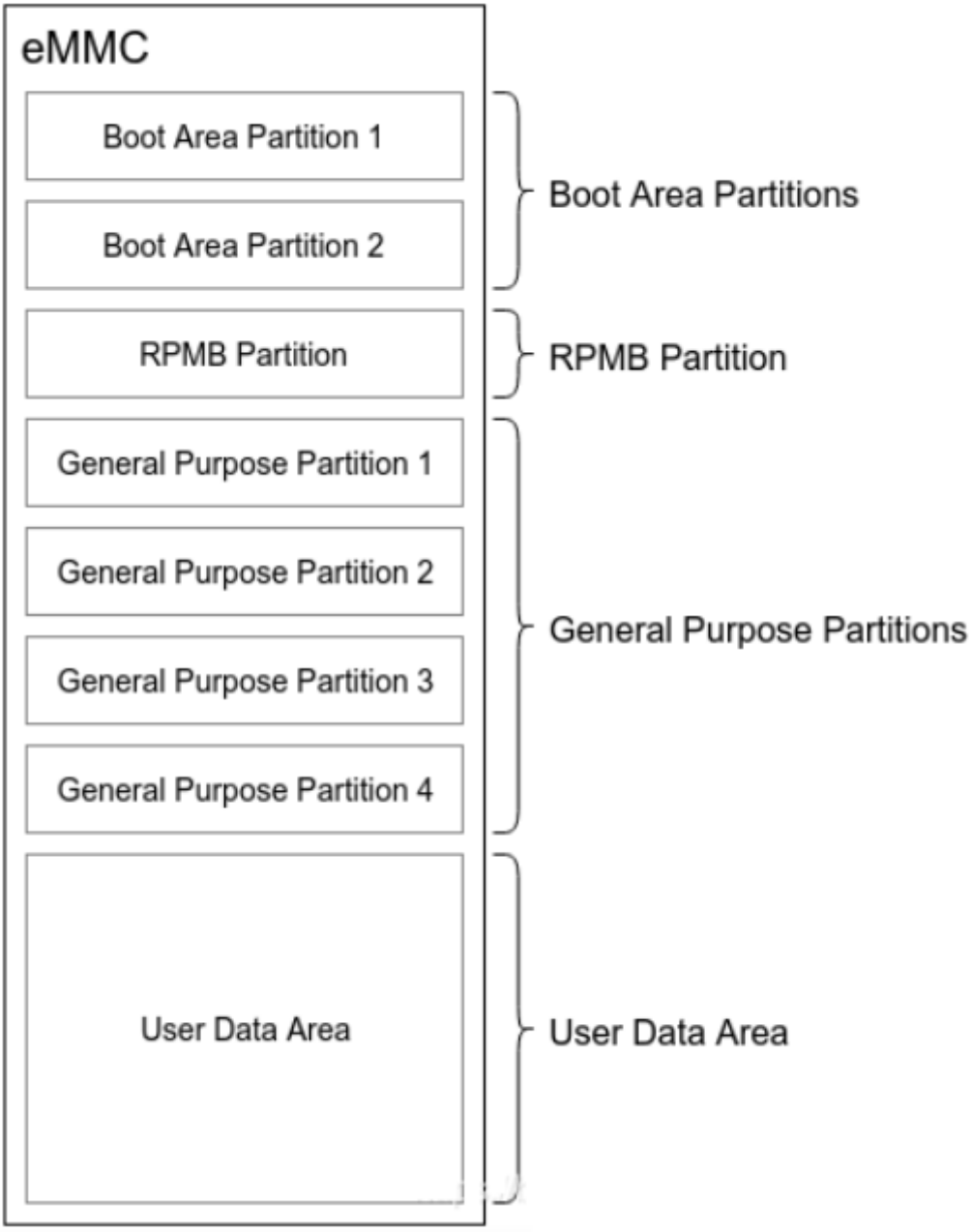
使用Busybox制作根文件系统

百问网出品

1. Linux系统启动流程

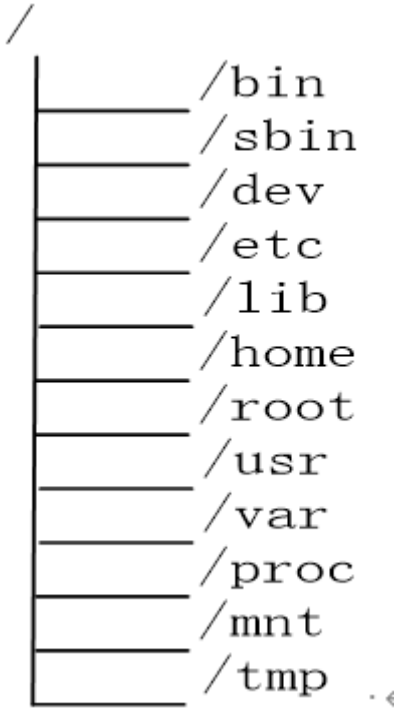


2. EMMC布局



3. 根文件系统内容

3.1 目录结构



3.2 内容

- /sbin/init：内核永远是其他它作为第一个进程
- /sbin/init的作用
  - 读配置文件：/etc/inittab
  - 根据配置文件启动其他APP

4. 手工制作SystemV根文件系统

4.1 编译busybox

```
tar xjf busybox-1.31.1.tar.bz2
cd busybox-1.31.1/
vi Makefile  :  CROSS_COMPILE ?= arm-buildroot-linux-gnueabihf-
make menuconfig
make
make CONFIG_PREFIX=tmp install
cd tmp/
ls
```

4.2 制作启动脚本

```
$ cd ~/create_my_fs/busybox-1.31.1/tmp/dev
$ sudo mknod console c 5 1
$ sudo mknod null c 1 3
```

```
book@100ask:~/create_my_image/busybox-1.31.1/tmp$
book@100ask:~/create_my_image/busybox-1.31.1/tmp$
book@100ask:~/create_my_image/busybox-1.31.1/tmp$ cat etc/inittab
console::askfirst:-/bin/sh
book@100ask:~/create_my_image/busybox-1.31.1/tmp$
book@100ask:~/create_my_image/busybox-1.31.1/tmp$ ls -l dev/
total 0
crw-r--r-- 1 root root 5, 1 Apr 29 09:02 console
crw-r--r-- 1 root root 1, 3 Apr 29 09:02 null
```

4.2 复制内核和设备树

```
$ cd ~/create_my_fs/busybox-1.31.1/tmp/
$ mkdir boot
$ cp ~/100ask_imx6ull-sdk/Linux-4.9.88/arch/arm/boot/zImage boot/
$ cp ~/100ask_imx6ull-sdk/Linux-4.9.88/arch/arm/boot/dts/100ask_imx6ull-
14x14.dtb boot/
```

4.3 复制lib

确定lib目录在哪：

```
$ echo "void main(){}" | arm-buildroot-linux-gnueabihf-gcc -E -v -
```

确定需要哪些库：

```
arm-buildroot-linux-gnueabihf-readelf -d bin/busybox | grep NEEDED
 0x00000001 (NEEDED)                     Shared library: [libm.so.6]
 0x00000001 (NEEDED)                     Shared library: [libresolv.so.2]
 0x00000001 (NEEDED)                     Shared library: [libc.so.6]
```

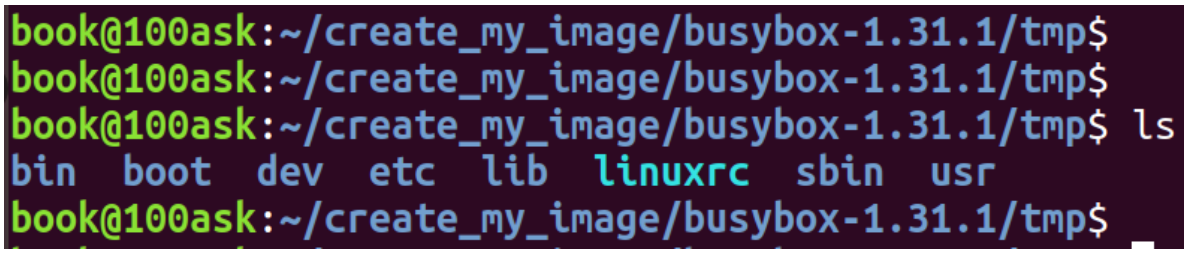
复制最少的lib：

```
$ mkdir lib
$ cp /home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-
buildroot/arm-buildroot-linux-gnueabihf/sysroot/lib/libm.so.6 lib/ -d
$ cp /home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-
buildroot/arm-buildroot-linux-gnueabihf/sysroot/lib/libm-2.30.so lib/ -d
$ cp /home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-
buildroot/arm-buildroot-linux-gnueabihf/sysroot/lib/libresolv*so* lib -d
$ cp /home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-
buildroot/arm-buildroot-linux-gnueabihf/sysroot/lib/libc.so.6 lib -d
$ cp /home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-
buildroot/arm-buildroot-linux-gnueabihf/sysroot/lib/libc-2.30.so lib -d
```

复制动态连接器：

```
$ cp /home/book/100ask_imx6ull-sdk/ToolChain/arm-buildroot-linux-gnueabihf_sdk-buildroot/arm-buildroot-linux-gnueabihf/sysroot/lib/ld-*  lib -d
```

4.3 制作镜像文件



4.3.1 genimage用法

命令结构

```
genimage \
  --rootpath "${ROOTPATH_TMP}"    \
  --tmppath "${GENIMAGE_TMP}"     \
  --inputpath "${BINARIES_DIR}"   \
  --outputpath "${BINARIES_DIR}"  \
  --config "genimage.cfg"
```

参数详解

参数	说明
--rootpath	**根文件系统路径**：配置文件里使用mountpoint指定的目录就是基于它，即rootpath/mountpoint
`--tmppath`	**临时文件路径**：生成镜像过程中临时文件的存放目录（如分区镜像、挂载点等），生成后自动清理。
--inputpath	**输入文件路径**：配置文件里使用file、files、image等参数指定文件时，这些文件来自于这个路径
--outputpath	**输出镜像路径**：最终生成的镜像文件（`.img`）和日志文件的存放目录。
--config	**配置文件路径**：指定 `genimage` 的配置文件（`.cfg`），定义镜像的分区结构、文件填充规则等。

4.3.2 配置文件示例

假设 `${GENIMAGE_CFG}` 内容如下：

```
image  emmc.img {
  hdimage {               // Generates DOS partition images.

  }

  partition rootfs-1 {
    partition-type = 0xC
    bootable = "yes"
    image = "boot.vfat"   // 这个分区里直接写入映像文件boot.vfat
    size = 10M            // 分区大小为10M
  }
  partition rootfs-2 {
    partition-type = 0x83
    bootable = "yes"
    image = "myrootfs.ext4" // 这个分区里直接写入映像文件myrootfs.ext4
    size = 1000M            // 分区大小为100M
  }
}

image  boot.vfat {
    vfat {                  // 制作vfat格式的映像文件
        files = { "u-boot-dtb.imx" }   // 内容来自inputpath指定的目录里的u-boot-
dtb.imx
    }
    size = 10M              // 生成的镜像文件大小
}

image myrootfs.ext4 {
    ext4 {}               // 制作ext4格式的映像文件
    name = "root"
    size = 100M           // 生成的镜像文件大小
    mountpoint = "/"      // 内容来自"rootpath"/{mountpoint}即 "rootpath"/
}
```

4.3.3 制作镜像文件的命令

```
genimage \
        --rootpath "/home/book/create_my_fs/busybox-1.31.1/tmp"     \
        --tmppath "/home/book/create_my_fs/mytmp"    \
        --inputpath "/home/book/create_my_fs/input"  \
        --outputpath "/home/book/create_my_fs/output" \
        --config "genimage.cfg"
```

在outpu目录先生成了emmc.img。

4.4 完善文件系统

按照/sbin/init进程运行流程添加各类配置文件。

4.4.1 /etc/inittab

/etc/inittab文件中每个条目用来定义一个子进程，并确定它的启动方法。格式如下：

```
<id>:<runlevels>:<action>:<process>
```

例如：

```
ttySAC0::askfirst:-/bin/sh
```

对于Busybox init程序，上述各个字段作用如下：

①： 表示这个子进程要使用的控制台(即标准输入、标准输出、标准错误设备)。如果省略，则使用与init进程一样的控制台。

②： 对于Busybox init程序，这个字段没有意义，可以省略。

③： 表示init进程如何控制这个子进程，有如表17.6所示的8种取值：

/etc/inittab文件中字段的意义

action名称	执行条件	说明
sysinit	系统启动后最先执行	只执行一次，init进程等待它结束才继续执行其他动作
wait	系统执行完sysinit进程后	只执行一次，init进程等待它结束才继续执行其他动作
once	系统执行完wait进程后	只执行一次，init进程不等待它结束
respawn	启动完once进程后	init进程监测发现子进程退出时，重新启动它
askfirst	启动完respawn进程后	与respawn类似，不过init进程先输出“Please press Enter to activate this console.”，等用户输入回车键之后才启动子进程。
shutdown	当系统关机时	即重启、关闭系统命令时
restart	Busybox中配置了CONFIG_FEATURE_USE_INITTAB，并且init进程接收到SIGHUP信号时	先重新读取、解析/etc/inittab文件，再执行restart程序
ctrlaltdel	按下Ctrl+Alt+Delete组合键时	

④： 要执行的程序，它可以是可执行程序，也可以是脚本。

如果字段前有“-”字符，这个程序被称为“交互的”。

在/etc/inittab文件文件的控制下，init进程的行为总结如下：

① 在系统启动前期，init进程首先启动为sysinit、wait、once的3类子进程。

② 在系统正常运行期间，init进程首先启动为respawn、askfirst的两类子进程，并监视它们，发现某个子进程退出时重新启动它。

③ 在系统退出时，执行为shutdown、restart、ctrlaltdel的3类子进程(之一或全部)。

如果根文件系统中没有/etc/inittab文件，Busybox init程序将使用如下默认的inittab条目：

```
::sysinit:/etc/init.d/rcS
::askfirst:/bin/sh
tty2::askfirst:/bin/sh
tty3::askfirst:/bin/sh
tty4::askfirst:/bin/sh
::ctrlaltdel:/sbin/reboot
::shutdown:/sbin/swapoff -a
::shutdown:/bin/umount -a -r
::restart:/sbin/init
```

（2）/etc/inittab实例。

仿照Busybox的examples/inittab文件，创建一个inittab文件，内容如下：

```
# /etc/inittab
# 这是init进程启动的第一个子进程，它是一个脚本，可以在里面指定用户想执行的操作
# 比如挂接其他文件系统、配置网络等
::sysinit:/etc/init.d/rcS

# 启动shell，以/dev/ttySAC0作为控制台
ttySAC0::askfirst:-/bin/sh

# 按下Ctrl+Alt+Delete之后执行的程序，不过在串口控制台中无法输入Ctrl+Alt+Delete组合键
::ctrlaltdel:/sbin/reboot

# 重启、关机前执行的程序
::shutdown:/bin/umount -a -r
```

4.4.2 /etc/fstab

内容如下，表示执行“mount -a”命令后将挂接proc、tmpfs文件系统：

```
# device   mount-point  type  options    dump fsck order
proc      /proc    proc  defaults    0   0
tmpfs     /tmp     tmpfs defaults    0   0
sys       /sys     sysfs defaults    0   0
devtmpfs  /dev     devtmpfs defaults    0   0
```

/etc/fstab文件被用来定义文件系统的“静态信息”，这些信息被用来控制mount命令的行为。文件中各字段意义如下：

① device：要挂接的设备

比如/dev/hda2、/dev/mtdblock1等设备文件；也可以是其他格式，比如对于proc文件系统这个字段没有意义，可以是任意值；对于NFS文件系统，这个字段为:

。

② mount-point：挂接点

③ type：文件系统类型

比如proc、jffs2、yaffs、ext2、nfs等；也可以是auto，表示自动检测文件系统类型。

④ options：挂接参数，以逗号隔开。

/etc/fstab的作用不仅仅是用来控制“mount -a”的行为，即使是一般的mount命令也受它控制，这可以从表17.8的参数看到。除与文件系统类型相关的参数外，常用的有以下几种取值：

/etc/fstab参数字段常用的取值

参数名	说明	默认值
auto noauto	决定执行“mount -a”时是否自动挂接。 auto：挂接；noauto：不挂接	auto
user nouser	user：允许普通用户挂接设备； nouser：只允许root用户挂接设备	nouser
exec noexec	exec：允许运行所挂接设备上的程序 noexec：不允许运行所挂接设备上的程序	exec
Ro	以只读方式挂接文件系统	
rw	以读写方式挂接文件系统	
sync async	sync：修改文件时，它会同步写入设备中； async：不会同步写入	sync
defaults	rw、suid、dev、exec、auto、nouser、async等的组合	

⑤ dump和fsck order：用来决定控制dump、fsck程序的行为。

dump是一个用来备份文件的程序，fsck是一个用来检查磁盘的程序。要想了解更多信息，请阅读它们的man手册。

dump程序根据dump字段的值来决定这个文件系统是否需要备份，如果没有这个字段，或其值为0，则dump程序忽略这个文件系统。

fsck程序根据fsck order字段来决定磁盘的检查顺序，一般来说对于根文件系统这个字段设为1，其他文件系统设为2。如果设为0，则fsck程序忽略这个文件系统。

4.4.3 /etc/init.d/rcS

这是一个脚本文件，可以在里面添加想自动执行命令。以下命令挂载/etc/fstab指定的文件系统：

```
#!/bin/sh
mount -a
echo /sbin/mdev > /proc/sys/kernel/hotplug
mdev -s
```

第一行表示这是一个脚本文件，运行时使用/bin/sh解析。

第二行用来配置IP地址。

第三行挂接/etc/fstab文件指定的所有文件系统。

最后，还要改变它的属性，使它能够执行：

```
chmod +x /etc/init.d/rcS
```

4.4.4 mdev

/etc/mdev.conf的格式：

```
<device regex> <uid>:<gid> <octal permissions> [<@|$|*> <command>]

device regex：正则表达式，表示哪一个设备
uid：owner
gid：组ID
octal permissions：以八进制表示的属性
@：创建设备节点之后执行命令
$：删除设备节点之前执行命令
*：创建设备节点之后 和 删除设备节点之前 执行命令
command：要执行的命令
```

正则表达式：

```
. 表示任意字符(换行符除外)
* 重复0次或多次
+ 重复1次或多次
? 重复0次或1次
[...]：这些字符之一
```

示例：

```
# /etc/mdev.conf 示例1：U盘自动加载
sda[1-9]+ 0:0 777 * if [ $ACTION = "add" ]; then mount /dev/$MDEV /mnt;
else umount /mnt; fi

# /etc/mdev.conf 示例2：U盘自动加载
sda[1-9]+ 0:0 777 * /bin/add_remove_udisk.sh

# /bin/add_remove_udisk.sh
#!/bin/sh
if [ $ACTION = "add" ];
then
    mount /dev/$MDEV /mnt;
else
    umount /mnt;
fi
```